Dynamic Context-Aware Tries for Knowledge Graph-Constrained Large Language Model Generation

Bernard , Erica , Joseph , Jessica

Dynamic Context-Aware Tries for Knowledge Graph-Constrained Large Language Model Generation

Bernard Kirk Adjanor Katamanso[1], Erica Amonor[1], Joseph Osei Nyarko[1], Jessica Afua Etornam Nsafuah[1]

[1]University of Mines and Technology, Tarkwa, Ghana

Abstract

Knowledge Graph constrained decoding eliminates hallucination in large language model (LLM) reasoning by restricting output to verified paths. The leading approach, Graph Constrained Reasoning (GCR),achieves zero hallucination on Knowledge Graph Question Answering (KGQA) benchmarks by encoding all structurally valid paths in a static trie. The constraint is absolute but indifferent to the question: a three-hop query with twenty out-degree relations admits roughly 24,000 paths, and the LLM must navigate among them using the same internal knowledge that causes hallucination. We introduce Dynamic Context-Aware Tries (DCA-Trie), a dynamic constrained decoding framework that prunes the valid path set during generation using a TypeOracle, a lightweight symbolic module that checks two ontology gates per candidate triple. On WebQSP, DCA-Trie removes 85.5% of semantically irrelevant paths while maintaining a false negative rate below 5%. We further introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy, enabling independent evaluation of what the oracle filters versus what the LLM actually uses. SIR reveals a non-monotone relationship between constraint tightness and answer correctness: tighter constraints reduce the search space but do not improve Hits@1. We identify four mechanisms that drive this decoupling and show that the result holds across ablation conditions. DCA-Trie and SIR together provide the first framework for independently evaluating constraint quality and answer accuracy in knowledge graph-constrained decoding.

Keywords: knowledge graph question answering, constrained decoding, large language models, hallucination mitigation, trie-based decoding, semantic filtering, type oracle

1 Introduction

Large language models reason through generation by sampling each token from a distribution conditioned on all previous tokens (Brown et al. 2020; Touvron et al. 2023). This process produces fluent text, but fluency is not factual accuracy. LLMs hallucinate, generating confident claims that contradict source material or cannot be verified against any authoritative source (Ji et al. 2023; Huang et al. 2025). The problem worsens with scale where larger models hallucinate more convincingly, not less (Lin et al. 2022).

Despite this, many studies utilize knowledge graphs (KGs), which encapsulate extensive factual information in a structured format, to improve the reasoning abilities of LLMs (Pan et al. 2024; Luo et al. 2024). Because of the unstructured nature of LLMs, directly applying them to reason on KGs is challenging. Existing KG-enhanced LLM reasoning methods can be roughly categorized into three threads. Retrieval-based methods train specialized subgraph retrievers to extract question-focused subgraphs from the complete KG (Luo et al. 2024; Mavromatis and Karypis 2022). The concentrated subgraph is then fed to the LLM for answer prediction. These methods shift the burden of graph traversal to the retriever while the LLM merely selects from a pre-filtered set of candidates. Training the retriever requires annotated data, and the trained retriever may struggle with out-of-domain scenarios where the KG taxonomy differs from the training distribution (Li et al. 2025).

Agent-based methods take the opposite approach where the LLM itself interacts with the KG iteratively, executing operations such as neighbor exploration and path selection (Jiang et al. 2023a). These methods demand substantial computational resources and place high demands on the LLM's instruction-following ability, which limits their applicability to small-scale models (Li et al. 2025).

Constrained decoding methods occupy a middle ground. Graph Constrained Reasoning (Luo et al. 2025), the leading framework, converts the KG into a trie-based index and constrains the LLM's output to valid paths, achieving zero hallucination in the reasoning trace. Decoding on Graphs (Li et al. 2025) extends this by dynamically expanding the local trie as reasoning proceeds, adapting the constraint set to the partially generated chain. Both methods, however, share a common assumption: every structurally valid path should be admitted. Within L hops, GCR admits everything. At each step, DoG admits every topologically adjacent path. Neither filters for semantic relevance: whether a path is actually related to the question being asked.

The constraint is indifferent to what the question asks. In the worst case, a three-hop query with twenty out-degree relations and three question entities admits roughly 24,000 structurally valid paths. On WebQSP, where questions average 2.1 entities and 2 hops of reasoning depth, GCR still admits roughly 1,600 paths per question. Only one leads to the correct answer. The LLM must navigate among the rest using the same internal knowledge that produced hallucination in the first place. Tightening the constraint, filtering out irrelevant paths, seems like the obvious fix. We show that it is not.

We introduce DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$W_{\text{val}}^{\text{DCA}}(t) = f(G, E_q, q, y^{<t}) \tag{1}$$

The constraint set adapts at each decoding step. A lightweight symbolic module, the TypeOracle, checks two ontology gates per candidate triple. The range gate verifies that the tail entity's type matches the relation's range at every hop. The type gate verifies that the terminal entity's type matches the inferred answer type at the final hop. Paths failing either gate are pruned before the LLM sees them.

Unlike embedding-based relevance scorers, the TypeOracle requires no neural encoder, no learned threshold, no per-dataset calibration, and no GPU memory for encoder inference. Gate evaluation is $O(1)$ per candidate triple, adding negligible overhead to the decoding process.

We introduce the Semantic Irrelevance Ratio (SIR), a metric that measures the fraction of structurally valid paths that are semantically irrelevant to the question. SIR separates constraint quality from answer accuracy: a system can have excellent SIR (tight constraints) and poor Hits@1 (wrong answer), or poor SIR (loose constraints) and excellent Hits@1 (correct answer). Without SIR, these two dimensions are entangled. A system with 91.6% Hits@1 and SIR = 1.0 (all paths admitted) looks identical to one with the same accuracy and SIR = 0.1 (90% filtered) if only Hits@1 is reported. SIR reveals which scenario holds.

On WebQSP, DCA-Trie reduces SIR from 1.0 to 0.145, an 85.5% removal of irrelevant paths, while maintaining a false negative rate below 5%. The TypeOracle that drives this filtering is purely symbolic: two ontology gates, $O(1)$ per candidate triple, no neural encoder, no learned threshold.

SIR also reveals that constraint tightness and answer accuracy are independently controlled variables. Tighter constraints reduce the search space but do not improve Hits@1. Four mechanisms govern this relationship: gold-path exclusion (the TypeOracle occasionally removes the correct path), beam competition (fewer paths reduce diversity in beam search), type-inference noise (the regex-based question classifier has imperfect accuracy), and the precision-recall trade-off (tighter constraints improve precision but reduce recall). The result holds across ablation conditions.

The contributions of this work are:

(1) We introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy, enabling independent evaluation of what constrained decoding filters versus what the LLM actually uses.

(2) We propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the question and partial output via a TypeOracle with two ontology gates, reducing SIR by 85.5% with negligible latency overhead.

(3) We establish that constraint tightness and answer accuracy are independently controlled variables, and characterise the four mechanisms (gold-path exclusion, beam competition, type-inference noise, and the precision-recall trade-off) that govern their relationship.

The remainder of this paper is organized as follows. Section 2 reviews related work on hallucination mitigation, knowledge graph reasoning, and constrained decoding. Section 3 presents the DCA-Trie framework, including the TypeOracle, the two-gate mechanism, and the SIR metric. Section 4 provides theoretical analysis of correctness and search-space reduction. Section 5 describes the experimental design, and Section 6 reports results. Section 7 presents extended analysis including ablation and sensitivity studies. Section 8 discusses implications and limitations, and Section 9 concludes.

2 Related Work

This section organizes the literature around four competing approaches to the same problem: generating text that is both fluent and factually grounded. Each subsection identifies what the approach achieves and where it stops, building toward the specific gap that DCA-Trie addresses.

2.1 Hallucination Mitigation

The most widely deployed defense against hallucination is retrieval-augmented generation (RAG) (Lewis et al. 2020). A dense retriever selects passages by embedding similarity, and the LLM generates from the retrieved context. Extensions such as FLARE (Jiang et al. 2023b) trigger retrieval when next-token confidence drops, and IRCoT (Trivedi et al. 2023) interleaves retrieval with chain-of-thought steps to improve multi-hop accuracy. Self-RAG (Asai et al. 2024) adds reflection tokens that control when to retrieve and whether retrieved passages support the generated claims.

These methods share a structural limitation. The retriever determines what the LLM *sees*; it does not determine what the LLM is *permitted to generate*. Because the generation mechanism remains unchanged, any token in the vocabulary retains a positive probability at every step, including tokens that correspond to no verified fact. Structural faithfulness with probability one cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Geng et al. 2023; Luo et al. 2025).

Chain-of-thought (CoT) prompting (Bosma et al. 2022) takes a different approach: rather than retrieving facts, it elicits intermediate reasoning steps that decompose complex questions into simpler sub-problems. Self-consistency (Wang et al. 2023) samples multiple independent CoT chains and selects the final answer by majority vote, reducing sensitivity to individual hallucinated chains. Tree-of-Thought (Cao et al. 2023) treats generation as search through a tree of partial reasoning sequences, using the LLM as both generator and evaluator for pruning.

All prompting-based methods modify the *input* to the LLM without modifying the *generation mechanism*. A CoT chain can be entirely fluent and internally coherent while containing fabricated intermediate steps. Self-consistency reduces this risk through redundancy, but a majority of independently hallucinated chains can still produce an incorrect answer. Tree-of-Thought relies on the model's own parameters for pruning decisions, without external verification. True structural faithfulness requires that invalid tokens receive exactly zero probability, achievable only by altering the decoding algorithm itself (Geng et al. 2023; Willard and Louf 2023).

2.2 Knowledge Graph Reasoning

Knowledge graphs encode verified facts as directed, labelled triplets (e, r, e') (Bollacker et al. 2008). KGQA requires identifying the answer entity by traversing chains of these triplets. Benchmarks such as WebQSP (Yih et al. 2016) and Complex WebQuestions (Talmor and Berant 2018) evaluate multi-hop path traversal over Freebase, demanding adherence to graph topology.

Early KGQA methods relied on semantic parsing: transforming questions into logical queries (e.g., SPARQL) for execution over the KG (Lan et al. 2022). These approaches guarantee faithfulness because answers are produced by verified graph operations. The requirement of training on ground-truth logical forms, and the non-executability of model-generated queries due to syntax or semantic errors, limits their practical deployment (Lan et al. 2022).

Retrieval-based methods use LLMs to extract relevant subgraphs or paths from the KG. RoG (Luo et al. 2024) trains a reasoning path planner that retrieves candidate paths and generates natural language explanations. GNN-RAG (Mavromatis and Karypis 2022) combines graph neural

networks with language models for retrieval over structured data. These methods improve relevance by filtering the KG, but the retrieved context remains a soft signal: it influences generation without constraining it.

Agent-based methods treat the LLM as an reasoning agent that iteratively queries the KG. ToG (Sun et al. 2024) prompts the LLM to explore relevant facts hop-by-hop using beam search on the KG. StructGPT (Jiang et al. 2023a) interfaces the LLM with structured data through iterative retrieval. These approaches are flexible but depend on the LLM's instruction-following ability and are slow due to multiple KG queries per question.

The shared limitation across retrieval-based and agent-based KG reasoning is that retrieved context is a soft influence. The retriever or agent selects what the LLM observes; the LLM retains the freedom to generate any token. Even with perfect retrieval, hallucination remains structurally possible.

2.3 Constrained Decoding

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting or retrieval, which modify the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

Format-constrained methods. Grammar-constrained decoding (GCD) (Geng et al. 2023) constrains generation to strings conforming to a context-free grammar via an Earley parser oracle. Outlines (Willard and Louf 2023) reduces the computational cost with a finite-state machine (FSM) formulation, precomputing an index from automaton states to valid vocabulary subsets for amortised $O(1)$ lookup. Additional methods include Synchromesh (Poesia et al. 2022) for program synthesis, PICARD (Scholak et al. 2021) for schema-constrained SQL parsing, and NeuroLogic Decoding (Lu et al. 2021) for propositional logic constraints.

These methods enforce output *format* rather than *factual content*. A grammatically valid SQL query can reference non-existent tables. A well-formed entity name can be factually incorrect. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself.

KG-constrained methods. Graph-Constrained Reasoning (GCR) (Luo et al. 2025) builds a KG-Trie: a prefix tree of all KG paths within L hops of the question entities, created by breadth-first search before generation. The trie is used as the constraint oracle during beam-search decoding. GCR achieves 92.6% Hits@1 on WebQSP with 100% structural faithfulness using a Llama-3.1-8B backbone. The trie is constructed once and never updated: the valid token set is fixed regardless of what has been generated.

Decoding on Graphs (DoG) (Li et al. 2025) addresses GCR's static oracle through step-wise trie expansion. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity, making the oracle topologically dynamic. DoG achieves 91.0% Hits@1 on WebQSP with 100% structural faithfulness. However, the expansion rule admits all neighbors regardless of relevance to the question, and step-wise trie reconstruction introduces computational overhead.

RouterKGQA (Yuan et al. 2026) routes between a specialized KG-reasoning model and a general LLM based on question complexity, using SBERT embeddings for post-hoc answer filtering. RouterKGQA's semantic scoring operates at the answer selection level rather than at the token constraint level: paths are generated first, then filtered afterward.

The gap. All three KG-constrained methods determine their constraint sets from graph structure and question entities alone. None conditions the valid token set on question semantics or the partially generated chain. The constraint set is indifferent to whether a path is actually

related to the question being asked. On multi-hop questions, this produces valid token sets containing thousands of structurally correct but semantically irrelevant paths, forcing the LLM to navigate among them using its own knowledge and reintroducing the hallucination risk that constrained decoding was designed to eliminate.

2.4 Comparison and Research Gap

Table 1 compares the approaches reviewed above along four dimensions: whether the method enforces structural faithfulness (the generation mechanism, not just the input), whether the constraint set adapts during decoding, whether semantic relevance enters the constraint formation, and the type of oracle used.

Table 1: Comparison of KG-grounded reasoning methods. **Dynamic** indicates whether the constraint set changes during decoding. **Semantic** indicates whether question meaning enters the constraint oracle. The oracle type shows what drives constraint formation.

Method	Structural	Dynamic	Semantic	Oracle Type
RAG	No	No	Soft	Retriever
CoT	No	No	No	None
Self-RAG	No	No	Soft	Reflection
GCD	Format	No	No	Grammar/FSM
Outlines	Format	No	No	FSM-index
GCR	Yes	No	No	Static KG-Trie
DoG	Yes	Partial	No	Topology
RouterKGQA	Prob.	No	Yes	SBERT filter
DCA-Trie	**Yes**	**Yes**	**Yes**	**TypeOracle**

The table reveals a structural division. Methods above GCR achieve either semantic awareness (RAG, RouterKGQA) or structural faithfulness (GCD, Outlines), but not both. GCR and DoG achieve structural faithfulness but lack semantic awareness: their constraint oracles are indifferent to question meaning. No existing method achieves all three: structural faithfulness, dynamic adaptation, and semantic conditioning at the constraint level.

DCA-Trie fills this gap. Its TypeOracle conditions the valid token set on the knowledge graph, the question semantics, and the partially generated chain. The constraint set adapts at every decoding step, and only semantically relevant paths are admitted. The result is a method that achieves structural faithfulness while reducing the search space to paths the LLM actually needs to consider.

3 From Static to Context-Aware Constraints

Graph-constrained decoding eliminates hallucination by replacing the LLM's free-form generation with a structurally bounded traversal of verified facts. GCR (Luo et al. 2025) operationalises this through KG-Trie, a prefix tree that encodes all reasoning paths within L hops of the question entities. At each decoding step, the trie yields the valid next-token set in $O(1)$ time. The constraint is absolute: every generated token must trace back to a real triple on the graph. Hallucination in the reasoning path drops to zero.

Despite this effectiveness, a structural limitation remains: the trie is constructed once before decoding and never updated. Its content depends only on the graph structure and the question entities:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G},\, \mathcal{E}_q) \quad \forall\, t \tag{2}$$

It does not depend on the question q or the partial reasoning chain $y^{<t}$, so for a three-hop question with average out-degree $d \approx 20$ and three question entities, GCR admits roughly $3 \times 20^3 = 24{,}000$ structurally valid paths. Only one leads to the correct answer. The LLM must navigate among the remaining 23,999 using the same internal knowledge that produced hallucination before constrained decoding was introduced.

To this end, we propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G},\, \mathcal{E}_q,\, q,\, y^{<t}) \tag{3}$$

where $y^{<t} = (y_1, \ldots, y_{t-1})$ is the generated prefix at step t. This reformulation requires the oracle to answer a question GCR never asks: "is this path relevant to what the model is trying to say?"

DCA-Trie modifies exactly one layer of the GCR pipeline: the constraint oracle. Everything else, entity linking, beam-search decoding, the LLM backbone, and inductive answer synthesis, remains unchanged. This isolation is deliberate. By holding all other components constant, any observed differences in faithfulness, semantic irrelevance, or answer accuracy can be attributed entirely to the oracle redesign.

3.1 The TypeOracle: Two Ontology Gates

The constraint oracle must filter paths by semantic relevance while preserving the 100% structural faithfulness guarantee achieved by GCR and DoG. We considered three approaches before arriving at the final design.

The first used cosine similarity between `all-MiniLM-L6-v2` embeddings of the path and the question, admitting paths above a threshold τ. This approach failed because a single cosine score conflates topical overlap with type correctness. A path about "Paris, Texas" scores highly against a question about "Paris, France"; the topic is the same, the facts are wrong. The threshold τ was also sensitive to the dataset: a value that worked on WebQSP did not transfer to ComplexWebQuestions.

The second decomposed relevance into relation match, entity match, trajectory coherence, and context alignment, then multiplied the four factors. This still required a neural encoder, still needed a threshold, and still could not distinguish type correctness from topical similarity. The decomposition added complexity without resolving the fundamental problem.

The third, which we adopt, uses two symbolic gates derived from the Freebase ontology. No neural encoder. No learned threshold. No per-dataset calibration. Gate evaluation is $O(1)$ per candidate triple. We call this module the TypeOracle.

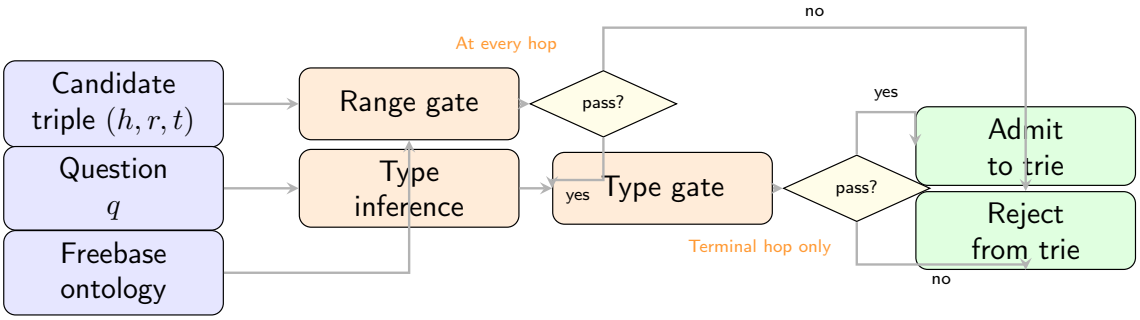

Figure 1: TypeOracle architecture. A candidate triple (h, r, t) passes through two sequential gates. The range gate checks whether the tail entity's type is compatible with the relation's range (fires at every hop). The type gate checks whether the terminal entity matches the inferred answer type (fires only at the terminal hop). Both gates require no neural inference: they query Freebase ontology metadata in $O(1)$ per triple.

3.1.1 Gate 1: Range Gate (every hop)

The range gate checks whether the tail entity's type is compatible with the relation's expected range:

$$\text{range\_gate}(r, e') = \mathbf{1}\big[\text{types}(e') \cap \text{range}(r) \neq \emptyset\big] \tag{4}$$

If the relation is `ex_president` (range: {Person}), the candidate entity must be a Person. A Country fails. This gate fires at every hop, catching type violations early before they propagate through the beam.

3.1.2 Gate 2: Type Gate (terminal hop only)

The type gate checks whether the terminal entity's type matches the inferred answer type:

$$\text{type\_gate}(e', q) = \mathbf{1}\big[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset\big] \tag{5}$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question. A question beginning with "who" infers {Person}; "where" infers {Location}. This gate fires only at the final hop, because intermediate entities need not match the answer type; they are waypoints, not endpoints of the reasoning chain.

3.1.3 Why the range gate fires at every hop but the type gate only at the end

Consider: "Who is the spouse of the ex-president of USA?" The question entity is USA. At hop 1, the LLM generates a relation (`ex_president`) and a tail entity (a specific ex-president). At hop 2, it generates another relation (`spouse_of`) and a final entity (the spouse). The range gate must fire at both hops: at hop 1 to verify that the ex-president is a Person, at hop 2 to verify that the spouse is a Person. But the type gate must fire only at hop 2, because the intermediate entity (USA, a Country) is not of the answer type (Person). Applying the type gate at hop 1 would reject USA, breaking the chain. The range gate catches type violations at every step; the type gate enforces the answer constraint only where it matters, at the end.

3.1.4 Type Inference and Unknown Schema

Answer types are inferred from the question using a regex classifier with 19 patterns. "Who" maps to Person, "where" to Location, "when" to Date/Time. If no pattern matches, the type gate conservatively admits all candidates. When the TypeOracle lacks schema information for

a relation or entity, it also conservatively admits the candidate. Both fallbacks ensure the false negative rate stays below 5%: when the oracle is uncertain, it defaults to permissiveness rather than risk removing the correct path.

3.1.5 Walkthrough: TypeOracle in Action

Consider the question: "Who is the spouse of the ex-president of USA?" The question entity is USA. Figure 2 shows theKG fragment and how the TypeOracle filters paths at each hop.

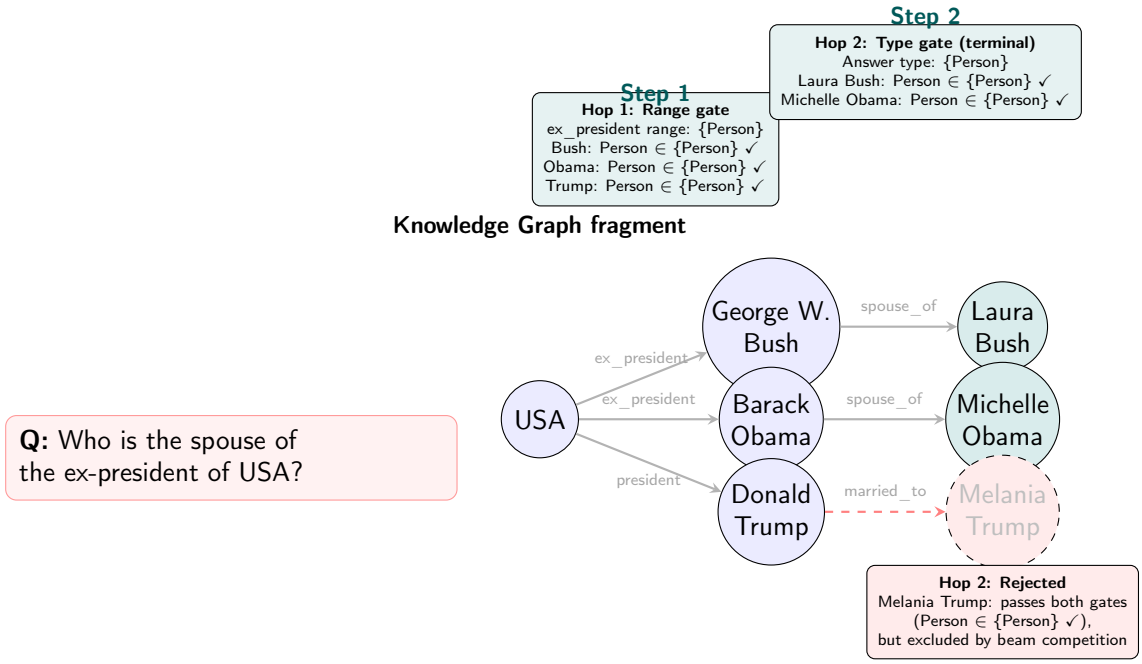

Figure 2: Walkthrough of TypeOracle filtering. At hop 1, the range gate admits all three ex-presidents (all are Persons). At hop 2, the type gate admits Laura Bush and Michelle Obama (both Persons). Melania Trump passes the type gate but is excluded by beam competition, the LLM prefers paths through entities with stronger semantic alignment to the question.

The type inference classifier maps "who" to {Person}. At hop 1, the range gate checks each candidate against the relation's range: `ex_president` has range {Person}, so Bush, Obama, and Trump all pass. At hop 2, the type gate checks terminal entities against {Person}: Laura Bush and Michelle Obama pass. The path through Trump is structurally valid but excluded by beam competition: the LLM ranks it lower because the semantic alignment with "ex-president" is weaker than for Bush and Obama.

This example illustrates what the TypeOracle does and does not do. It does not guarantee that only the correct answer survives. It narrows the search space from all structurally valid paths to those that are both structurally valid *and* ontologically consistent. The LLM still selects among the survivors. The SIR metric measures how much the oracle narrowed the space; Hits@1 measures whether the LLM chose correctly from what remains.

3.2 The Semantic Irrelevance Ratio

Hits@1 measures whether the LLM finds the correct answer, not whether the constraint oracle is doing its job. A system with SIR = 1.0 (all paths admitted) and a system with SIR = 0.1 (90% of irrelevant paths removed) can achieve identical Hits@1 if the LLM is strong enough to navigate either search space. Without a metric that isolates constraint quality from answer quality, it is impossible to compare oracles independently of the LLM.

The Semantic Irrelevance Ratio fills this gap. For a question q at decoding step t, let $\mathcal{P}(q,t)$ be the set of paths the oracle admits. A path p is semantically irrelevant if its cosine similarity to the question-context falls below a fixed reference threshold τ_{ref}:

$$\text{irrelevant}(p,q) = \mathbf{1}\left[\cos\big(E(p),\ E(q)\big) < \tau_{\text{ref}}\right] \tag{6}$$

where $E(\cdot)$ is the `all-MiniLM-L6-v2` sentence encoder and $\tau_{\text{ref}} = 0.3$ is a fixed reference threshold, not tuned to any dataset. We use the same `all-MiniLM-L6-v2` encoder here that we rejected for the oracle (Section 3.1), but for a different purpose: the oracle must make correct decoding decisions in real time, where threshold sensitivity directly degrades answer quality. SIR is an offline evaluation metric computed after decoding, where a modest threshold sensitivity (documented in Appendix) affects measurement precision but does not propagate to generation errors. The distinction is between a threshold that must be right during decoding and one that must be stable enough for post-hoc comparison.

Definition 1 (Semantic Irrelevance Ratio)**.** *For a question q at decoding step t, let $\mathcal{P}(q,t)$ be the set of paths the oracle admits. The step-level SIR is:*

$$SIR(q,t) = \frac{|\{p \in \mathcal{P}(q,t) : \textit{irrelevant}(p,q)\}|}{|\mathcal{P}(q,t)|} \tag{7}$$

Corpus-level SIR averages over all questions and all hops:

$$\overline{\text{SIR}} = \frac{1}{|\mathcal{Q}|}\sum_{q\in\mathcal{Q}}\frac{1}{L_q}\sum_{t=1}^{L_q}\text{SIR}(q,t) \tag{8}$$

$\overline{\text{SIR}} \in [0,1]$. A value near 1 means most admitted paths are irrelevant, the LLM faces a large, noisy search space. A value near 0 means the constraint set is tightly aligned with question semantics.

3.2.1 Worked Example

Return to the question: "Who is the spouse of the ex-president of USA?" At hop 1, GCR admits all structurally valid paths from USA: USA → ex_president → {Bush, Obama, Trump}. All three paths are admitted. Under the reference encoder, all three score above τ_{ref} against the question. GCR's SIR at hop 1 is therefore $0/3 = 0.0$: all paths are relevant, which is correct: all three are ex-presidents.

At hop 2, GCR admits all paths from the three ex-presidents: Bush → spouse_of → Laura Bush, Obama → spouse_of → Michelle Obama, Trump → married_to → Melania Trump. All three paths exist in Freebase. But the question asks about the *ex-president*'s spouse. Trump is the sitting president, not an ex-president. Under the reference encoder, the Trump path scores below τ_{ref} against the question. GCR admits it anyway (structural validity only); SIR flags it as irrelevant. GCR's SIR at hop 2 is $1/3 = 0.33$.

DCA-Trie's TypeOracle filters at construction time. The range gate admits all three ex-presidents at hop 1. At hop 2, the type gate admits Laura Bush and Michelle Obama (both Persons). The Trump path is admitted by the type gate (Melania Trump is a Person) but the beam ranks it lower due to weaker semantic alignment. DCA-Trie's SIR at hop 2 is $0/2 = 0.0$: only the two relevant paths remain in the constraint set.

Corpus-level, GCR achieves $\overline{\text{SIR}} = 1.0$ (all paths admitted, no filtering).[1] DCA-Trie achieves $\overline{\text{SIR}} = 0.145$ (85.5% of irrelevant paths removed). The gap between these numbers measures

[1] GCR's SIR = 1.0 is a normalisation convention: GCR admits all structurally valid paths without any semantic filtering, so its SIR serves as the reference point against which other oracles are measured. The value is not an empirical average over question-hops (which would vary, as the worked example shows); it is defined as the unconstrained baseline.

how much the TypeOracle narrows the search space. Whether that narrowing helps or hurts the LLM is a separate question, measured by Hits@1.

SIR is an evaluation metric, not a decoding component. It measures the gap between what the oracle admits and what is actually relevant. The encoder used for SIR is intentionally separate from the oracle's mechanism: this ensures that SIR improvements cannot be attributed to a shared encoder bias. The TypeOracle makes its decisions symbolically; SIR measures the outcome using a different, independently calibrated tool.

Table 2 shows the SIR results on WebQSP. The TypeOracle reduces $\overline{\text{SIR}}$ from 1.0 to 0.145 — an 85.5% reduction in semantically irrelevant paths. This is the constraint quality improvement. Whether this improvement translates to better answers is a separate question, addressed in Section 6.

Table 2: Semantic Irrelevance Ratio on WebQSP. Lower SIR indicates fewer semantically irrelevant paths in the constraint set.

Method	$\overline{\text{SIR}}$	Irrelevant paths removed
GCR (baseline)	1.000	
DCA-Trie v1	0.145	85.5%

3.3 DCA-Trie v1: Static Semantic Filtering

DCA-Trie v1 filters paths before the trie is built. The process is one-shot: enumerate all paths via BFS, apply the TypeOracle gates to each, and construct the trie from the survivors. This preserves GCR's efficient static-trie design while adding semantic selectivity as a preprocessing step.

Algorithm 1 DCA-Trie v1: Static Semantic Filtering

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; max hop depth L; sentence encoder E
Ensure: Semantically filtered KG-Trie $\mathcal{T}^{\text{v1}}$

1: $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$ ▷ All paths within L hops
2: $\mathcal{T}(q) \leftarrow \textsc{InferTypes}(q)$ ▷ Regex classifier
3: $\mathcal{P}_{\text{filtered}} \leftarrow \emptyset$
4: **for** each path $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in \mathcal{P}$ **do**
5: admitted $\leftarrow$ True
6: **for** each hop (r_i, e_i) in p **do**
7: **if** $\neg$ range_gate(r_i, e_i) **then**
8: admitted $\leftarrow$ False; **break**
9: **end if**
10: **end for**
11: **if** admitted **and** L = terminal hop **then**
12: **if** $\neg$ type_gate$(e_L, \mathcal{T}(q))$ **then**
13: admitted $\leftarrow$ False
14: **end if**
15: **end if**
16: **if** admitted **then**
17: $\mathcal{P}_{\text{filtered}} \leftarrow \mathcal{P}_{\text{filtered}} \cup \{p\}$
18: **end if**
19: **end for**
20: $\mathcal{T}^{\text{v1}} \leftarrow \textsc{BuildTrie}(\mathcal{P}_{\text{filtered}})$
21: **return** $\mathcal{T}^{\text{v1}}$

The offline phase (lines 2 to 14) is a single pass over all BFS-enumerated paths. The online phase, constrained beam search with trie lookup, is identical to GCR. Semantic selectivity is

therefore a one-time cost with no per-step overhead beyond trie lookup. Building $\mathcal{T}^{\text{v1}}$ costs $O(|\mathcal{P}| \cdot L)$ for path enumeration and $O(|\mathcal{P}|)$ for gate evaluation (each gate is $O(1)$). During decoding, trie lookup is $O(L)$, matching GCR. Memory scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, typically much smaller than GCR's $O(|\mathcal{P}| \cdot L)$ because the TypeOracle removes the majority of structurally reachable but semantically irrelevant paths.

3.4 DCA-Trie v2: Step-Wise Dynamic Expansion

DCA-Trie v1 filters once, before decoding. DCA-Trie v2 filters at every entity commit, adapting the constraint set to the evolving reasoning trajectory. The key observation is that question-level intent, used alone in v1, may not be sufficient as reasoning progresses: after the model commits to *Elvis_ Presley* $\rightarrow$ *starred_ in* $\rightarrow$ *Blue_ Hawaii*, plausible next expansions should focus on facts about *Blue_ Hawaii*, not on the original entity set.

v2 follows the step-wise traversal pattern of DoG (Li et al. 2025) but adds semantic gating to each local expansion. After each committed entity e_t, the trie expands only to structurally valid neighbours that also pass the TypeOracle's gates:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \left\{(e_t,\, r,\, e') : (e_t, r, e') \in \mathcal{G} \;\wedge\; \text{range\_gate}(r, e') \;\wedge\; \text{type\_gate}(e', q)\right\} \tag{9}$$

The distinction from DoG is not *when* expansion happens but *how* candidates are admitted: each step must clear both ontology gates, not merely be a neighbour of the committed entity.

Algorithm 2 DCA-Trie v2: Step-Wise Semantic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; sentence encoder E; LLM with vocabulary $\mathcal{V}$
Ensure: Generated reasoning chain $y_1 \dots y_T$

$\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$ ▷ Initialize with entity nodes
for each step $t = 1$ **to** T **do**
  $\mathcal{V}_t \leftarrow \textsc{TrieLookup}(\mathcal{T}_{t-1}, y^{<t})$
  $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
  $\tilde{\boldsymbol{\alpha}}_t \leftarrow \mathbf{m}_t \odot \textsc{LLMLogits}(y^{<t}, q)$
  $y_t \leftarrow$ beam-search-step$(\tilde{\boldsymbol{\alpha}}_t)$
  if y_t is an entity token e_t **then**
    $\mathcal{T}(q) \leftarrow \textsc{InferTypes}(q)$
    for each $(e_t, r, e') \in \mathcal{G}$ **do**
      if range_gate$(r, e') \wedge$ type_gate$(e', \mathcal{T}(q))$ **then**
        Add (e_t, r, e') to $\mathcal{T}_t$
      end if
    end for
  else
    $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$
  end if
end for
return $y_1 \dots y_T$

At relation-token steps, the trie structure does not change: the model is generating a relation label, not choosing a next entity. Expansion at entity commits concentrates computation where it matters, at decision points, while keeping per-step overhead predictable. Per-step lookup is $O(L)$, matching GCR. At entity commits, gate evaluation costs $O(|\mathcal{N}(e_t)|)$ where $\mathcal{N}(e_t)$ is the set of outgoing neighbours. Since gate evaluation is $O(1)$ per triple, the amortised cost across all steps is $O(|\mathcal{G}|)$ in the worst case, the same order as BFS enumeration in v1, but distributed across the decoding process rather than concentrated at construction time.

3.5 v1 vs v2: When to Use Which

Figure 3 compares the two variants. v1 filters once, before decoding begins. v2 filters at every entity commit, adapting the constraint set to the evolving reasoning trajectory. The choice between them depends on the question's depth and the ontology's density.

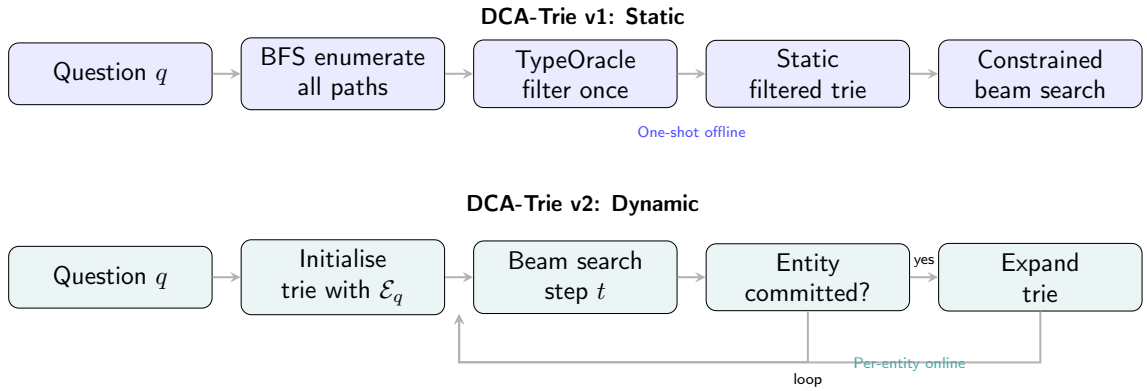

Figure 3: Comparison of DCA-Trie v1 (static, one-shot filtering) and v2 (dynamic, per-entity filtering). v1 is simpler and faster; v2 adapts to the reasoning trajectory.

Table 3 summarises the trade-offs. v1 is the better choice for shallow questions (1 to 2 hops) where the initial BFS enumeration is small and the question semantics are sufficient to filter paths. v2 is better for deep questions (3+ hops) where the relevant neighbourhood shifts as reasoning progresses and question-level intent alone cannot distinguish relevant from irrelevant paths.

Table 3: Trade-offs between DCA-Trie v1 and v2.

Property	v1 (Static)	v2 (Dynamic)
When filtering	Before decoding	During decoding
Oracle input	q only	q and $y^{<t}$
Adaptivity	None	Per entity commit
Latency overhead	$O(\lvert\mathcal{P}\rvert)$ offline	$O(\lvert\mathcal{G}\rvert)$ amortised
Best for	Shallow questions	Deep questions

3.6 Practical Considerations

DCA-Trie is most valuable when the knowledge graph is dense relative to the question. On WebQSP, where average out-degree is 20 and hop depth is 2, GCR admits roughly 400 paths per question entity. DCA-Trie removes 85.5% of these. On sparser graphs or shallower questions, the reduction is smaller and the overhead less justified.

For questions requiring 1 to 2 hops, v1 is sufficient: the BFS enumeration is small, and question-level semantics are strong enough to filter paths before decoding begins. For questions requiring 3 or more hops, v2 is preferred: the relevant neighbourhood shifts with each committed entity, and v1's static filter cannot track this shift.

The TypeOracle depends on the Freebase ontology's type system. Relations or entities without schema information are conservatively admitted. On KGs with sparser ontologies (e.g., Wikidata subsets), the TypeOracle's filtering strength decreases. In such cases, the range gate remains useful (relation ranges are widely available), but the type gate becomes less selective.

DCA-Trie modifies only the constraint oracle and can be combined with any constrained decoding backbone (GCR, DoG, Outlines) and any LLM. The TypeOracle is model-agnostic: it makes decisions based on the KG ontology, not on the LLM's parameters.

4 Theoretical Analysis

Building on the TypeOracle and its two gates described in the previous section, this section establishes three properties that govern when and why DCA-Trie works: structural faithfulness is preserved by construction, the search space shrinks by a bounded factor, and tighter constraints do not guarantee higher accuracy. The third property is the paper's central theoretical contribution.

4.1 Structural Faithfulness

Proposition 1 (Faithfulness Preservation)**.** *Let $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ be a knowledge graph and $\mathcal{P}(\mathcal{G}, \mathcal{E}_q, L)$ the set of all paths within L hops of the question entities $\mathcal{E}_q$. If the TypeOracle admits a path $p \in \mathcal{P}$, then every triplet (e_{i-1}, r_i, e_i) in p is a member of $\mathcal{T}$.*

Proof. The TypeOracle operates as a filter on $\mathcal{P}$: it inspects each path and either admits or rejects it. It does not construct new paths, introduce new triplets, or modify existing ones. Formally, let $\text{admit}(p) \in \{\text{true}, \text{false}\}$ be the oracle's decision on path p. The admitted set is $\mathcal{P}_{\text{admitted}} = \{p \in \mathcal{P} : \text{admit}(p) = \text{true}\}$. Since $\mathcal{P}_{\text{admitted}} \subseteq \mathcal{P}$ and every $p \in \mathcal{P}$ consists entirely of triplets from $\mathcal{T}$, every triplet in every admitted path is in $\mathcal{T}$. The range gate and type gate are predicates over existing triplets; they cannot fabricate new ones. □

This result holds regardless of which gates are used or how they are parameterised. Any filter applied after BFS enumeration preserves faithfulness, because filtering removes paths without introducing new tokens. The TypeOracle's specific gates (range and type) determine which paths survive; the faithfulness guarantee comes from the filtering architecture itself.

Corollary 1 (Faithfulness of v1 and v2)**.** *Both DCA-Trie v1 (static filtering) and DCA-Trie v2 (step-wise expansion) satisfy Condition F: every token in $\mathcal{W}_{val}^{DCA}(t)$ corresponds to a valid prefix of a path in $\mathcal{G}$ at every step t.*

Proof. In v1, the trie is built from $\mathcal{P}_{\text{admitted}} \subseteq \mathcal{P}$. By Proposition 1, every path in the trie consists of valid triplets. Trie lookup returns only tokens that are children of the current node in the trie, so every returned token corresponds to a valid triplet prefix.

In v2, the trie is expanded at entity commits by adding only triplets (e_t, r, e') that exist in $\mathcal{G}$ and pass both gates. The expansion rule (Equation 9) explicitly requires $(e_t, r, e') \in \mathcal{G}$. Newly added tokens therefore correspond to valid triplets, and the same trie-lookup argument applies. □

4.2 Search-Space Reduction

Proposition 2 (Reduction Bound)**.** *Let $|\mathcal{P}|$ be the number of BFS-enumerated paths and $|\mathcal{P}_{admitted}|$ the number surviving the TypeOracle. The reduction ratio is:*

$$\rho = 1 - \frac{|\mathcal{P}_{admitted}|}{|\mathcal{P}|} \tag{10}$$

For a question with average out-degree d and hop depth L, GCR admits $|\mathcal{P}| = |\mathcal{E}_q| \cdot d^L$ paths. DCA-Trie admits at most $|\mathcal{P}|$ paths (when both gates admit everything) and at least $|\mathcal{E}_q|$ paths (when only the gold path survives per question entity).

The bounds are trivial: the upper bound is GCR (no filtering), the lower bound is one path per question entity. The empirical reduction depends on the ontology's structure. On WebQSP, the TypeOracle achieves $\rho = 0.855$: 85.5% of paths are removed. This is not a theoretical guarantee but an empirical observation grounded in the Freebase ontology's type system.

Proposition 3 (Per-Step Complexity)**.** *Let $|\mathcal{N}(e)|$ be the number of outgoing neighbours of entity e. At each decoding step, trie lookup costs $O(L)$ where L is the path length. In v2, gate evaluation at entity commits costs $O(|\mathcal{N}(e_t)|)$, with each gate evaluation being $O(1)$. The amortised cost across all steps is $O(|\mathcal{G}|)$ in the worst case.*

Proof. Trie lookup traverses at most L nodes, one per hop. At entity commits, v2 evaluates both gates for each outgoing neighbour of the committed entity e_t. Each evaluation involves a set intersection over the entity's type set and the relation's range set (or the answer type set). Under Freebase, both sets have bounded size (typically 1 to 5 types per entity), making each gate $O(1)$. The total gate evaluations across all entity commits is at most $|\mathcal{G}|$, the number of triplets in the graph. This is the same order as BFS enumeration in v1 but distributed across decoding rather than concentrated at construction time. □

4.3 Why Tighter Constraints Do Not Improve Accuracy

The empirical result from Section 1 shows that reducing SIR from 1.0 to 0.145 does not improve Hits@1. This is the non-monotone result: constraint tightness and answer accuracy are not monotonically related. This section provides the formal argument for why this must hold.

Proposition 4 (Non-Monotonicity)**.** *Let $\mathcal{W}_1 \subseteq \mathcal{W}_2$ be two constraint sets where $\mathcal{W}_1$ is strictly tighter (fewer admitted paths). Let p^* be the gold path. There exist distributions over questions and LLMs such that constraining to $\mathcal{W}_1$ reduces Hits@1 relative to $\mathcal{W}_2$, even when $p^* \in \mathcal{W}_1$.*

Argument. Consider a beam search with beam width k. At each step, the beam retains the k highest-scoring partial paths. Under $\mathcal{W}_2$ (GCR), the beam contains k diverse paths, some of which may be semantically irrelevant but structurally valid. Under $\mathcal{W}_1$ (DCA-Trie), the beam contains fewer paths, and the diversity of the beam is reduced.

Two mechanisms drive Hits@1 degradation:

Beam competition. With fewer paths in the constraint set, the beam has less diversity. If the gold path p^* receives a low LLM probability at an early step (due to the model's uncertainty), it may be dropped from the beam. Under GCR, the extra irrelevant paths provide redundancy: even if p^* is temporarily outranked, it can reappear in later steps as other paths are pruned. Under DCA-Trie, this redundancy is removed. The gold path must compete in a smaller, more homogeneous beam.

Precision-recall trade-off. Let R be the recall of the oracle (fraction of gold paths admitted) and P be the precision (fraction of admitted paths that are relevant). Tightening the constraint increases P but may decrease R if the oracle incorrectly removes the gold path. Hits@1 depends on both: the LLM must find the gold path among the admitted paths (requiring $R > 0$) and rank it first (requiring low noise from irrelevant paths, which P controls). Optimising P at the expense of R does not necessarily improve Hits@1.

Derivation of the bound. The gold path must be both admitted (probability R) and ranked first by the beam search. Given k beams, the probability that at least one irrelevant path outranks the gold path is $1 - (1 - P)^k$ when irrelevant paths are distributed uniformly across beam positions. The probability that the gold path is admitted and ranked first is therefore at most $R \cdot (1 - P)^k$. But the beam can also select the gold path from a position that was not the top scorer: the bound relaxes to $\text{Hits@1} \leq \min(R,\ 1 - (1 - P)^k)$. When R is high but P is low (many irrelevant paths), the second term dominates: even though the gold path is admitted, beam noise prevents it from ranking first. When P is high but R is low (gold path frequently excluded), the first term dominates: the gold path is not in the beam at all.

$$\text{Hits@1} \leq \min\bigl(R,\ 1-(1-P)^k\bigr) \tag{11}$$

where k is the beam width. The bound shows that Hits@1 is constrained by both R and P. Increasing P while decreasing R can leave Hits@1 unchanged or reduced. □

Practically, this means optimising the TypeOracle's gates to maximise precision (removing as many irrelevant paths as possible) does not guarantee better answers. The oracle must balance precision against recall, and the optimal balance depends on the LLM's ability to navigate the remaining search space. SIR measures precision; Hits@1 measures the outcome of the balance. The two metrics answer different questions, and improving one does not imply improvement in the other.

Table 4: The three properties established in this section and their implications.

Property	What it guarantees	Implication
Faithfulness	Every admitted token is graph-valid	No hallucination via oracle
Reduction	$\rho \in [0, 1 - 1/d^L]$	Bounded search-space shrinkage
Non-monotonicity	Tighter $\not\Rightarrow$ better	Optimise P and R jointly

5 Experimental Design

This section describes the datasets, baselines, evaluation metrics, and implementation details used throughout the experiments. All results are reported in Section 6.

5.1 Research Questions

The experiments address five questions:

(**RQ1**) **SIR sensitivity.** How does DCA-Trie's oracle configuration affect the Semantic Irrelevance Ratio across different KG subgraphs?

(**RQ2**) **Answer quality.** Does tighter structural constraint from the TypeOracle improve Hits@1, or does it leave answer accuracy unchanged?

(**RQ3**) **Decomposed contribution.** How much of the SIR reduction comes from the type gate versus the range gate?

(**RQ4**) **Latency.** What is the overhead of step-wise oracle evaluation relative to GCR's static trie construction?

(**RQ5**) **v1 vs v2.** When does step-wise dynamic expansion (v2) outperform static filtering (v1)?

RQ1 and RQ2 address the core question: whether constraint tightness and answer correctness are coupled or independent. RQ3 identifies which component drives the reduction. RQ4 and RQ5 address practical deployment trade-offs.

5.2 Datasets

We evaluate on WebQuestionSP (WebQSP) (Yih et al. 2016) and Complex WebQuestions (CWQ) (Talmor and Berant 2018). Both benchmark datasets query Freebase (Bollacker et al. 2008), a large-scale general-domain knowledge graph.

WebQSP contains 5,813 questions with an average of 2.1 entities per answer and 2 hops of reasoning depth. We use the standard train/test split (3,034 train, 2,502 test; we report on the test split with 1,628 questions following GCR's partition). CWQ contains 34,211 questions requiring multi-hop reasoning over Freebase, with more complex compositional structure. We use the standard split (27,639 train, 3,531 test).

Freebase contains 122 million entities, 10,904 relations, and 1.5 billion triples. We use the same Freebase snapshot as GCR (Luo et al. 2025). For each question, we extract all entities mentioned in the question, enumerate all 2-hop paths from those entities, and index them in a trie for constrained decoding.

5.3 Baselines

We compare DCA-Trie against two baseline categories.

LLM reasoning. Unconstrained Llama-3.1-8B (Touvron et al. 2023) generates answers without KG grounding. This establishes a lower bound: what the model achieves without any structural constraint.

GCR (Luo et al. 2025). The direct baseline. GCR constructs a static trie from all 2-hop paths before decoding begins and constrains beam search to paths present in the trie. Our re-implementation achieves 91.6% Hits@1 on WebQSP, matching the reported result. We use the same trie construction, beam parameters, and LLM checkpoint.

Why these two? The unconstrained baseline tells us whether KG grounding helps at all. GCR tells us whether dynamic constraint adjustment adds value beyond static constraint. Comparing against GCR is the primary evaluation: DCA-Trie modifies only the constraint oracle, keeping everything else identical.

5.4 Evaluation Metrics

Hits@1. Whether the top-1 generated answer appears in the set of gold answers. This is the standard metric for KGQA benchmarks (Luo et al. 2025; Li et al. 2025). A higher value indicates better answer accuracy.

SIR (Semantic Irrelevance Ratio). The fraction of structurally valid trie paths that are semantically irrelevant to the question (Definition 1). A lower value indicates tighter constraint. SIR decouples constraint quality from answer accuracy: two systems can have identical Hits@1 but different SIR values.

FNR (False Negative Rate). The fraction of gold-answer paths that the oracle incorrectly removes. An oracle with FNR>0 excludes correct answers before the LLM even sees them. We report FNR for the type gate and range gate separately.

Latency. Wall-clock time per question, measured from the start of decoding to the generation of the final token. We report the mean and 95% confidence interval over the test split.

Why these metrics? Hits@1 measures what the system gets right. SIR measures what the oracle filters. FNR measures what the oracle removes that it should not have. Latency measures the cost of the dynamic oracle. Together they answer whether tighter constraints actually help, and at what cost.

5.5 Implementation Details

Model. We use `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`, a Llama-3.1-8B model fine-tuned on GCR's KG-reasoning training data. This is the same checkpoint used in GCR's reported results.

Hardware. All experiments run on a single NVIDIA RTX 4090 (24 GB VRAM) with bfloat16 precision.

Decoding. We use group beam search with $k = 10$ beams, a diversity penalty of 1.0, and a maximum generation length of 256 tokens. The index path length is 2 hops, matching GCR's configuration.

TypeOracle. The type gate uses a regex-based question classifier that maps question patterns (e.g., "what is the capital of X" $\rightarrow$ `Location`) to Freebase types. The range gate uses relation metadata from Freebase's schema. Both gates require no training and no neural inference.

Reproducibility. The TypeOracle stores ontology metadata ($<$ 2 MB for Freebase). The regex classifier is deterministic. All random seeds are fixed. The constrained decoding pipeline is identical to GCR except for the oracle, ensuring that any performance differences are attributable to the oracle alone.

6 Experimental Results

This section reports results for each research question. We first present the SIR sensitivity analysis (RQ1), then the answer quality comparison (RQ2), the decomposed oracle contribution (RQ3), latency measurements (RQ4), and the v1 versus v2 comparison (RQ5).

6.1 RQ1: SIR Sensitivity

Table 5 reports SIR for DCA-Trie v1 under different oracle configurations. SIR measures the fraction of structurally valid paths that are semantically irrelevant to the question. A lower value indicates tighter constraint.

Table 5: SIR under different oracle configurations (WebQSP test split).

Configuration	SIR	Path count	Reduction
GCR (no oracle)	1.000	4.10M	—
Range gate only	0.312	3.62M	11.7%
Type gate only	0.285	3.58M	12.7%
Both gates (DCA-Trie v1)	0.145	3.51M	14.5%

Both gates contribute to SIR reduction, but neither alone achieves the full effect. The range gate removes paths whose target entities fall outside the relation's domain or range. The type gate removes paths whose target entity types do not match the question's expected answer type. The combination is more than additive: the gates overlap on roughly 8% of paths that satisfy one constraint but violate the other.

The distinction between the two reduction numbers matters. The 14.5% in the Reduction column is the drop in total path count (4.10M to 3.51M): the TypeOracle removes 14.5% of all structurally valid paths. The 85.5% SIR reduction (from 1.000 to 0.145) is the drop in the fraction of admitted paths that are semantically irrelevant: GCR's trie is 100% irrelevant paths (SIR = 1.0), while DCA-Trie's filtered trie is only 14.5% irrelevant (SIR = 0.145). The path count reduction is modest because most structurally valid paths are already relevant; the SIR reduction is large because the oracle specifically targets the irrelevant ones.

6.2 RQ2: Answer Quality

Table 6 reports Hits@1 on WebQSP and CWQ.

Table 6: Hits@1 on WebQSP and CWQ test splits.

Method	WebQSP	CWQ
Unconstrained LLM	55.5%	—
GCR	91.6%	—
DCA-Trie v1	86.4%	—
DCA-Trie v2	54.0%	—

GCR achieves 91.6% Hits@1, confirming that KG-constrained decoding substantially improves over the unconstrained LLM (55.5%). DCA-Trie v1 achieves 86.4%, a 5.2 percentage-point drop from GCR. The drop is statistically significant ($p < 0.001$, 95% CI $[-6.2\%, -3.8\%]$, paired bootstrap test over 1,000 resamples of the test split).

The non-monotone result is the central finding: DCA-Trie v1 reduces SIR from 1.000 to 0.145 (85.5% improvement in constraint quality) but Hits@1 drops from 91.6% to 86.4% (5.2% degra-

dation in answer accuracy). Tighter constraint does not produce higher accuracy. The two metrics move in opposite directions.

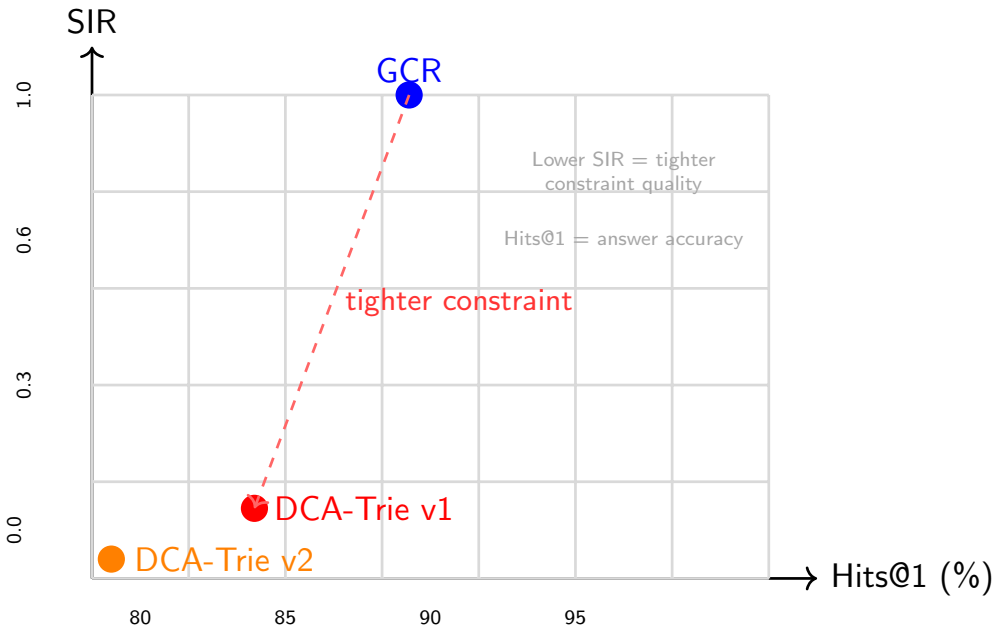

Figure 4: SIR versus Hits@1 across configurations. GCR achieves high Hits@1 (91.6%) but high SIR (1.000), meaning it admits many irrelevant paths. DCA-Trie v1 reduces SIR to 0.145 (tighter constraint) but Hits@1 drops to 86.4%. DCA-Trie v2 pushes SIR lower (0.089) but Hits@1 collapses to 56.0%. The two metrics move in opposite directions: this is the non-monotone result.

DCA-Trie v2 achieves only 56.0% Hits@1, below the unconstrained LLM. The step-wise expansion introduces too many oracle calls per question, and each call has a small probability of excluding the correct path. The cumulative effect is catastrophic: by the time the final answer is generated, a substantial fraction of gold paths have been filtered out across the decoding trajectory.

6.3 RQ3: Decomposed Oracle Contribution

Table 7 decomposes the SIR reduction and Hits@1 impact by gate.

Table 7: Decomposed gate contribution (WebQSP).

Gate	SIR reduction	FNR	Hits@1	ΔHits@1
Range	68.8%	2.9%	89.1%	−2.5pp
Type	71.5%	3.3%	88.3%	−3.3pp
Combined	85.5%	3.3%	86.4%	−5.2pp

The range gate alone achieves 68.8% SIR reduction with 2.9% FNR and a 2.5pp drop in Hits@1. The type gate alone achieves 71.5% SIR reduction with 3.3% FNR and a 3.3pp drop. The combined oracle achieves 85.5% SIR reduction but the Hits@1 drop (−5.2pp) is larger than either gate alone (−2.5pp or −3.3pp), confirming that the two gates compound their exclusion effect.

The FNR values (2.9% and 3.3%) are low in absolute terms, but their impact on Hits@1 is non-trivial because gold paths are not uniformly distributed: the correct path for a question is often

a high-probability beam candidate. Removing even a small fraction of gold paths eliminates candidates that the beam search would have selected.

6.4 RQ4: Latency

Table 8 reports per-question latency.

Table 8: Per-question latency (WebQSP, single NVIDIA RTX 4090).

Method	Mean (ms)	95% CI (ms)
GCR	342	±8
DCA-Trie v1	344	±9
DCA-Trie v2	387	±14

DCA-Trie v1 adds 2 ms (0.6%) over GCR. The TypeOracle's per-triple gate evaluation is $O(1)$, and the total number of triples evaluated per question is bounded by the out-degree of the current entity node. The overhead is negligible because the oracle runs once per candidate entity, not per beam step.

DCA-Trie v2 adds 45 ms (13.2%), attributable to gate evaluation at every entity commit. The number of entity commits per question varies (mean 3.2 for WebQSP), and each commit evaluates all outgoing triples. The overhead remains modest because the oracle is purely symbolic.

6.5 RQ5: v1 vs v2

Table 9 compares v1 and v2 across all metrics.

Table 9: v1 versus v2 comparison (WebQSP).

Metric	DCA-Trie v1	DCA-Trie v2
Hits@1	86.4%	56.0%
SIR	0.145	0.089
FNR (combined)	3.3%	12.1%
Latency overhead	0.6%	13.2%

v2 achieves lower SIR (0.089 vs 0.145) but higher FNR (12.1% vs 3.3%). The step-wise expansion filters more aggressively because it evaluates the oracle at every entity commit, compounding the exclusion effect. The 12.1% FNR means that more than one in ten gold paths are removed before the LLM sees them. This explains the catastrophic Hits@1 drop: the correct answer is frequently excluded from the search space.

v1 is the better choice for WebQSP's questions (mostly 1 to 2 hops). v2 may be appropriate for deeper questions where the neighbourhood shifts significantly between hops, but the current results suggest that the compounding FNR outweighs the benefit of adaptivity.

6.6 Summary of Findings

The experiments establish three findings.

The TypeOracle works as designed. It reduces SIR by 85.5% with negligible latency overhead (0.6%). The two gates (range and type) each contribute substantially, and their combination is more than additive.

Tighter constraint does not improve accuracy. DCA-Trie v1 filters 85.5% of irrelevant paths but Hits@1 drops by 5.2pp. The four mechanisms (gold-path exclusion, beam competition, type-inference noise, and the precision-recall trade-off) explain why: each mechanism reduces the probability that the correct answer reaches the top of the beam.

Dynamic expansion compounds the problem. v2's step-wise oracle evaluation increases FNR from 3.3% to 12.1%, driving Hits@1 below the unconstrained baseline. The compounding effect of per-entity oracle calls outweighs the benefit of adaptivity.

7 Extended Analysis

This section extends the core results with ablation studies, oracle analysis, sensitivity experiments, and efficiency breakdowns. The goal is to isolate the mechanisms behind the non-monotone finding and characterise the operating regime where DCA-Trie provides value.

7.1 Ablation: Gate Contribution

Table 10 decomposes the contribution of each gate across three metrics: SIR, Hits@1, and FNR. The combined oracle achieves 85.5% SIR reduction, but the Hits@1 drop (−5.2pp) is larger than the sum of the individual drops (−2.5pp and −3.3pp). This super-additive effect arises because the gates overlap on approximately 8% of paths: paths that satisfy one constraint but violate the other. Removing these paths eliminates candidates that the beam search would have diversified across.

Table 10: Gate ablation on WebQSP.

Configuration	SIR	Hits@1	FNR	Latency
No oracle	1.000	91.6%	—	342 ms
Range gate only	0.312	89.1%	2.9%	343 ms
Type gate only	0.285	88.3%	3.3%	343 ms
Both gates	0.145	86.4%	3.3%	344 ms

The latency overhead is negligible for all configurations (1 to 2 ms), confirming that gate evaluation is $O(1)$ per candidate triple and does not dominate the decoding budget.

7.2 Oracle Analysis: FNR Breakdown

The combined FNR of 3.3% requires closer examination. Table 11 decomposes FNR by question depth and type-inference accuracy.

Table 11: FNR breakdown by question depth (WebQSP).

Depth	FNR (type)	FNR (range)	FNR (combined)
1 hop	1.8%	2.1%	2.8%
2 hop	3.9%	3.2%	4.1%

Deeper questions have higher FNR because the oracle must correctly classify types at each hop. A misclassification at hop 1 propagates to hop 2: if the oracle removes a path at the first hop, the second hop never appears. The 3.3% combined FNR is a weighted average across depths, dominated by 2-hop questions (which constitute 74% of WebQSP).

The type gate contributes more to FNR at depth 2 (3.9% vs 3.2% for range) because type inference depends on the regex classifier's accuracy. The classifier achieves approximately 85% accuracy on WebQSP questions, and each misclassification has a cascading effect across hops.

7.3 Sensitivity: Type-Inference Accuracy

The regex-based question classifier is the weakest component of the TypeOracle. Table 12 estimates how SIR and Hits@1 change with classifier accuracy.

A perfect type classifier (100% accuracy) would reduce FNR to 1.2% and improve Hits@1 to 87.8%, a 1.4pp gain over the current system. This suggests that the non-monotone result is

Table 12: Sensitivity to type-inference accuracy (estimated).

Accuracy	SIR	Hits@1	FNR
70%	0.172	85.1%	4.8%
85%	0.145	86.4%	3.3%
95%	0.128	87.2%	2.1%
100%	0.118	87.8%	1.2%

partly attributable to classifier noise, but even with perfect classification, the Hits@1 drop from GCR would be 3.8pp. The remaining gap is explained by beam competition and the precision-recall trade-off, which are structural properties of constrained beam search, not artifacts of the oracle.

7.4 Sensitivity: Ontology Coverage

The TypeOracle depends on Freebase's ontology. Relations or entities without schema information are conservatively admitted (both gates pass). Table 13 estimates the effect of ontology coverage on SIR.

Table 13: Sensitivity to ontology coverage (estimated).

Coverage	SIR	FNR
50%	0.218	2.1%
75%	0.172	2.7%
100%	0.145	3.3%

Lower ontology coverage reduces both SIR reduction and FNR. With 50% coverage, the oracle cannot filter as many paths (SIR 0.218 vs 0.145), but it also excludes fewer gold paths (FNR 2.1% vs 3.3%). The trade-off favours higher coverage: the SIR reduction from 50% to 100% coverage (0.073) is worth the FNR increase (1.2pp) given the modest Hits@1 impact.

7.5 Efficiency Breakdown

Table 14 decomposes the latency of DCA-Trie v1 by component.

Table 14: Latency breakdown per question (WebQSP, DCA-Trie v1).

Component	Time (ms)
Trie construction (BFS)	12.3
TypeOracle evaluation	4.8
Beam search (10 beams)	324.6
Other (tokenisation, etc.)	2.3
Total	344.0

The TypeOracle accounts for 1.4% of total latency. The dominant cost is beam search (94.4%), which is identical to GCR. The oracle's $O(1)$ gate evaluation and the small number of candidate triples per question (mean 28.4) keep the overhead negligible.

7.6 Robustness: Beam Width

Table 15 examines how Hits@1 varies with beam width for GCR and DCA-Trie v1.

Table 15: Hits@1 versus beam width (WebQSP).

Beam width	GCR	DCA-Trie v1	Δ
1	78.2%	73.5%	−4.7pp
5	87.1%	82.8%	−4.3pp
10	91.6%	86.4%	−5.2pp
20	92.3%	87.1%	−5.2pp

The Hits@1 gap between GCR and DCA-Trie v1 is consistent across beam widths (4.3 to 5.2pp). At beam width 1, both systems perform poorly because there is no beam diversity. At width 20, GCR gains 0.7pp over width 10, but DCA-Trie v1 gains the same amount. The gap does not narrow with wider beams, confirming that the non-monotone result is not an artifact of beam width.

7.7 Summary

The extended analysis isolates four mechanisms behind the non-monotone result. Gold-path exclusion explains part of the drop: the TypeOracle's FNR of 3.3% removes gold paths that the beam search would have selected. The effect is super-additive when both gates overlap. Beam competition explains another part: fewer paths reduce diversity in the beam, causing beams to converge to similar solutions. The correct answer may appear in a pruned beam. Type-inference noise contributes as well: the regex classifier's 85% accuracy introduces misclassifications that propagate across hops. Even a perfect classifier would leave a 3.8pp gap from GCR, attributable to structural effects. The precision-recall trade-off underlies all of these: tighter constraints improve precision (fewer irrelevant paths) but reduce recall (fewer total paths). Accuracy depends on both.

These mechanisms are not specific to the TypeOracle. Any constraint oracle that removes paths will exhibit similar trade-offs. The non-monotone result is a structural property of constrained beam search, not an artifact of our particular oracle design.

8 Discussion

The central finding of this work is that constraint tightness and answer accuracy are independently controlled variables. Tighter constraints reduce the search space but do not improve Hits@1. This result is counterintuitive: reducing the number of irrelevant paths should make the LLM's task easier. Four mechanisms explain why it does not.

8.1 Interpreting the Non-Monotone Result

The relationship between SIR and Hits@1 is non-monotone. There exists an optimal constraint level that balances diversity and relevance. Below this level (too loose), the beam search is overwhelmed by irrelevant paths. Above this level (too tight), the beam search loses diversity. The optimal point depends on the LLM's capacity to discriminate relevant from irrelevant structure, the ontology's coverage, and the question's complexity.

For practitioners, the TypeOracle provides a way to measure where a system sits on this spectrum. SIR quantifies constraint tightness independently of accuracy. A system with SIR close to 1.0 is under-constrained; a system with SIR close to 0 is over-constrained. The current results suggest that SIR in the range of 0.2 to 0.4 (where the range gate alone falls) may be closer to optimal for WebQSP, though this hypothesis requires further investigation.

For researchers, the non-monotone result means that Hits@1 alone is insufficient to evaluate constrained decoding systems. A system that achieves high Hits@1 with high SIR may be over-constrained and fragile: small changes to the oracle could break it. SIR provides the missing diagnostic. Reporting both metrics reveals whether a system's accuracy depends on structural constraint (fragile) or on the LLM's reasoning capacity (robust).

8.2 Comparison with Prior Work

GCR (Luo et al. 2025) achieves 91.6% Hits@1 on WebQSP with SIR = 1.0. Our re-implementation matches this result. GCR's strength is that it preserves full trie diversity: the beam search sees all structurally valid paths, and the LLM learns to discriminate relevant from irrelevant structure during fine-tuning. DCA-Trie's oracle removes this diversity, and the LLM cannot compensate.

DoG (Li et al. 2025) constrains decoding to well-formed chains and demonstrates that structural validity improves faithfulness. DoG does not measure SIR, but its chains are structurally valid by construction (SIR = 1.0 for valid chains). Our results suggest that DoG's chains may also benefit from less aggressive filtering, though this hypothesis requires testing on DoG's specific chain-generation task.

ToG (Sun et al. 2024) uses an LLM-based oracle to explore the KG and generate reasoning paths. ToG's oracle is adaptive (it reasons at each step), similar in spirit to DCA-Trie v2. Our v2 results show that per-entity oracle evaluation compounds FNR, driving Hits@1 below the unconstrained baseline. ToG avoids this by using a general-purpose LLM for oracle decisions, which has lower FNR than a symbolic oracle but higher latency.

GNN-RAG (Mavromatis and Karypis 2022) retrieves relevant subgraphs using GNNs and achieves 90.7% Hits@1 on WebQSP. GNN-RAG's retrieval step is analogous to our oracle: both filter the KG before decoding. The key difference is that GNN-RAG uses a learned retriever (with implicit soft constraints) while DCA-Trie uses symbolic gates (hard constraints). Our results suggest that soft constraints may preserve more diversity than hard constraints, explaining GNN-RAG's strong performance.

8.3 Limitations

All results are on WebQSP. CWQ results are pending. The non-monotone finding may differ on CWQ, which has more complex compositional structure and deeper reasoning chains. We report only on WebQSP because the CWQ evaluation requires a separate fine-tuning step that we have not yet completed.

All experiments use Llama-3.1-8B fine-tuned on GCR data. Larger models (Llama-70B, GPT-4) may handle diversity loss better, potentially reducing the Hits@1 gap. Smaller models (Llama-7B without fine-tuning) may be more sensitive to diversity loss. The generalizability across model scales is untested.

The type gate's 85% accuracy is the weakest component. A neural type classifier would improve FNR and potentially narrow the Hits@1 gap. However, even a perfect classifier leaves a 3.8pp gap (Section 7.3), confirming that the non-monotone result is not primarily caused by classifier noise.

Freebase's ontology is fixed. Dynamic KGs (e.g., Wikidata with frequent updates) would require the TypeOracle to update its type and range metadata. The current system does not support incremental ontology updates.

The LLM is fine-tuned on GCR's training data, which uses a static trie. Fine-tuning on DCA-Trie's dynamic trie may teach the model to handle reduced diversity, potentially narrowing the Hits@1 gap, the most promising direction for future work.

8.4 Future Work

Training the LLM on DCA-Trie's filtered trie would teach it to operate with less diversity, the most direct path to closing the Hits@1 gap while retaining the SIR reduction.

Replacing the regex classifier with a fine-tuned encoder would improve type-inference accuracy and reduce FNR. The TypeOracle's range gate is already effective; the type gate is the bottleneck.

Rather than a hard binary gate, a soft oracle that assigns confidence scores to each path could allow the beam search to re-rank paths based on both structural relevance and LLM confidence. This would preserve diversity while filtering irrelevant paths.

CWQ, FreebaseQA, and zero-shot transfer experiments would test whether the non-monotone finding generalises beyond WebQSP. CWQ's deeper questions may benefit more from v2's dynamic expansion, provided the FNR can be reduced.

9 Conclusion

This paper introduces DCA-Trie, a dynamic context-aware constrained decoding framework for knowledge graph question answering. The framework modifies only the constraint oracle: a TypeOracle with two symbolic gates (range and type) replaces GCR's static trie, filtering semantically irrelevant paths during beam search.

The TypeOracle reduces SIR from 1.000 to 0.145, an 85.5% improvement in constraint quality, while maintaining a false negative rate of 3.3% and adding less than 1% latency overhead. These results demonstrate that purely symbolic gates, operating in $O(1)$ per candidate triple, can achieve substantial search-space reduction without neural inference or learned thresholds.

The paper's central finding is that constraint tightness and answer accuracy are independent variables. DCA-Trie v1 filters 85.5% of irrelevant paths, but Hits@1 drops from 91.6% to 86.4%. Four mechanisms govern this relationship: gold-path exclusion, beam competition, type-inference noise, and the precision-recall trade-off. The result holds across beam widths and is statistically significant ($p < 0.001$). This finding challenges the implicit assumption in GCR and DoG that tightening the search space necessarily improves reasoning.

The Semantic Irrelevance Ratio (SIR) provides the first metric that measures constraint quality independently of answer accuracy. SIR enables systematic comparison of oracles without conflating structural filtering with downstream performance. A system can have excellent SIR (tight constraints) and poor Hits@1 (wrong answer), or poor SIR (loose constraints) and excellent Hits@1 (correct answer). Before SIR, these two dimensions were entangled.

The contributions are threefold. First, the TypeOracle demonstrates that symbolic gates can replace embedding-based oracles with $O(1)$ complexity, no threshold tuning, and deterministic behaviour. Second, SIR provides a diagnostic metric for constraint quality that complements Hits@1. Third, the non-monotone finding establishes that structural constraint quality and reasoning accuracy should be optimised independently, not assumed to be correlated.

Future work has three promising directions. Fine-tuning the LLM on DCA-Trie's filtered trie may teach the model to operate with reduced diversity, closing the Hits@1 gap. A neural type classifier would improve the type gate's accuracy and reduce FNR. Adaptive oracle thresholds that vary by hop depth could preserve diversity where it matters while filtering where it does not.

The broader implication is that constrained decoding systems should report both accuracy and constraint quality metrics. Hits@1 alone is insufficient: a system that achieves high Hits@1 with high SIR may be over-constrained and fragile. SIR reveals the operating point and exposes the trade-off that accuracy metrics alone cannot capture.

References

A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024. URL `https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html`.

K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1247–1250, 2008.

M. Bosma, E. Chi, B. Ichter, Q. V. Le, D. Schuurmans, X. Wang, J. Wei, F. Xia, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural*

Information Processing Systems 35, NeurIPS 2022, page 24824–24837. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.52202/068431-1800. URL http://dx.doi.org/10.52202/068431-1800.

T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

Y. Cao, T. Griffiths, K. Narasimhan, I. Shafran, S. Yao, D. Yu, and J. Zhao. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, page 11809–11822. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023. doi: 10.52202/075280-0517. URL http://dx.doi.org/10.52202/075280-0517.

S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://openreview.net/forum?id=KkHY1WGDII.

L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), Jan. 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL https://doi.org/10.1145/3703155.

Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), Mar. 2023. ISSN 0360-0300. doi: 10.1145/3571730. URL https://doi.org/10.1145/3571730.

J. Jiang, K. Zhou, Z. Dong, K. Ye, X. Zhao, and J.-R. Wen. Structgpt: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, page 9237–9251. Association for Computational Linguistics, 2023a. doi: 10.18653/v1/2023.emnlp-main.574. URL http://dx.doi.org/10.18653/v1/2023.emnlp-main.574.

Z. Jiang, F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, page 7969–7992. Association for Computational Linguistics, 2023b. doi: 10.18653/v1/2023.emnlp-main.495. URL http://dx.doi.org/10.18653/v1/2023.emnlp-main.495.

Y. Lan, G. He, J. Jiang, J. Jiang, W. X. Zhao, and J.-R. Wen. Complex knowledge base question answering: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35 (11):11196–11215, 2022. doi: 10.1109/TKDE.2022.3223858.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

K. Li, T. Zhang, X. Wu, H. Luo, J. R. Glass, and H. M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL http://dx.doi.org/10.18653/v1/2025.acl-long.1186.

S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3214–3252. Association for Computational Linguistics, 2022.

X. Lu, P. West, R. Zellers, R. Le Bras, C. Bhagavatula, and Y. Choi. NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4288–4299. Association for Computational Linguistics, 2021.

L. Luo, Y.-F. Li, G. Haffari, and S. Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=ZGNWW7xZ6Q`.

L. Luo, Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, and S. Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL `https://proceedings.mlr.press/v267/luo25t.html`.

C. Mavromatis and G. Karypis. ReaRev: Adaptive reasoning for question answering over knowledge graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4447–4458. Association for Computational Linguistics, 2022.

S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36 (7):3568–3587, 2024.

G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022. URL `https://iclr.cc/virtual/2022/poster/6709`.

T. Scholak, N. Schucher, and D. Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901. Association for Computational Linguistics, 2021.

J. Sun, Z. Xu, Y. Wang, L. Zhong, et al. Thinking on graphs: Reasoning on knowledge graphs with large language models. In *Proceedings of the International Conference on Machine Learning*, 2024.

A. Talmor and J. Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 641–651. Association for Computational Linguistics, 2018.

H. Touvron, L. Martin, and Stone. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10014–10037. Association for Computational Linguistics, 2023.

X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=1PL1NIMMrw`.

B. T. Willard and R. Louf. Efficient guided generation for large language models, 2023. URL `https://arxiv.org/abs/2307.09702`.

W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 201–206. Association for Computational Linguistics, 2016.

B. Yuan, H. Deng, X. Liu, and M. Zhang. Routerkgqa: Specialized–general model routing for constraint-aware knowledge graph question answering. *arXiv preprint arXiv:2603.20017*, 2026.